

A Comparison of Voronoi Fracturing in Vulkan and OpenGL

1st Samuel Lloyd

Student at Falmouth University Games Academy

Falmouth University

Penryn, UK

samlloyd2005@gmail.com

Abstract—

I. INTRODUCTION

As destruction effects in games have grown in usage, complexity and scale, the need for performance has grown in tandem [31] [18] [27] [16]. As such, choosing the right graphics API to optimise for hardware has become increasingly important. A commonly used algorithm for destruction is Voronoi fracturing [52], where a mesh is sliced against a generated Voronoi diagram. This algorithm was selected because it is a popular algorithm used in many applications such as the "Chaos" system from the popular game engine Unreal engine 5. Similarly, it is used in several games well known for their destruction effects like Battlefield [16]. Both the Voronoi diagram generation and clipping steps have been well researched algorithmically [51] [52] [14] [39] but few formal studies benchmark for this algorithm across different graphics APIs. Graphics APIs can differ in the performance they offer due to differing levels of control [30] and what GPU architecture they were designed for [8]. This is useful when deciding which API to use for an application featuring this algorithm and when deciding what API to teach based on its modern relevance. The choice of API may also make Voronoi more accessible to those with lower-end hardware as the hardware barrier could be lowered. This paper aims to provide clear benchmarking results using Vulkan (for its modernity) and OpenGL (for its viability on older machines). Beginning with background, discussing existing literature, alternative algorithms, and how Voronoi fracturing works. This then leads into the main research question and the hypotheses. The following section details the methodology chosen for testing which graphics API performs better during Voronoi fracturing. A technical description of the software artefact is given, along with the techniques used to validate and generate the data. A plan of how the data is to be safely stored and an analysis of said data using Python's "pandas" library is presented. Following this, a social, ethical, professional, legal and sustainability analysis of the project is provided. Finally, the results are compiled, analysed, and discussed.

II. BACKGROUND

A. Graphics APIs

In graphics programming, graphics APIs are software abstractions that allow applications to send commands to the GPU [20], providing functions such as rendering images to a window [42]. These APIs are standard function interfaces with the exact implementations of these functions left to the GPU vendors or driver providers [1]. This results in different GPUs with similar capabilities showing different performances and different bugs when using the same API. There are many different types of graphics APIs but widely used game engines such as Unreal Engine and Unity make use of Vulkan [24], OpenGL [22], DirectX [45], and Metal [5]. Unity allows you to switch the choice of API to use [50] while Unreal Engine uses a rendering hardware interface (RHI) to translate code into the appropriate language [19].

B. Performance with Graphics APIs

OpenGL (the predecessor to Vulkan) has an easier learning curve and provides an easier development experience at the cost of control. This often results in lower performance [30]. Vulkan is a more modern API designed to address some of OpenGL's shortcomings such as not being able to take advantage of the multi-threading capabilities of modern GPUs [8]. It is an explicit API that allows for more explicit control of the GPU [23]. As the API can be customised to fit an application, Vulkan often allows better performance. This does not mean that Vulkan is always more performant as an inefficient implementation of Vulkan can lead to OpenGL performing better [8]. Vulkan is claimed to have 'close to metal control' that allows for high-performance on a variety of hardware and operating systems. However, as it is based on the Mantle codebase, Vulkan often shows better performance on AMD GPUs [3].

C. Destruction in Games

1) *Voronoi*: The main algorithm this paper looks at is the Voronoi diagram. These are named after Georgy Voronoy who published "New applications of continuous parameters to the theory of quadratic forms" [51] where he described regions of space defined by inequalities, later coming to be known as Voronoi diagrams. A Voronoi diagram is a way of splitting a shape by the areas that are closest to a point inside of the

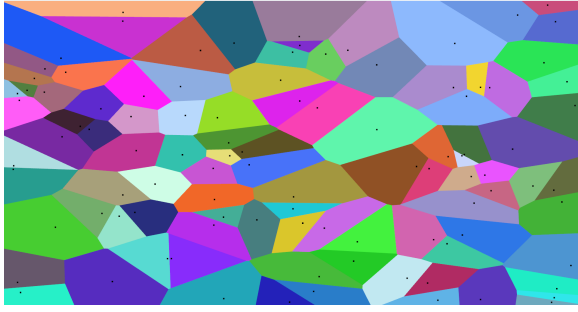


Fig. 1. Voronoi generation in 2D, created in OpenGL

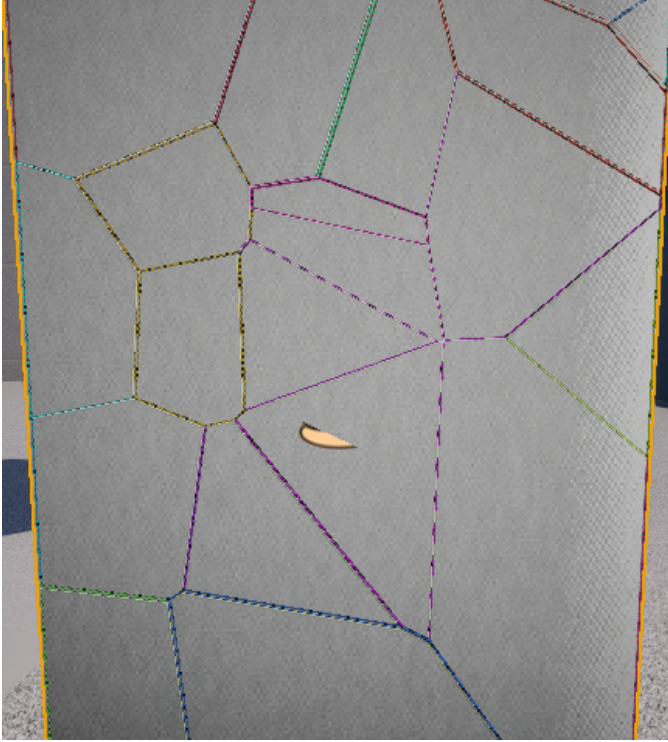


Fig. 2. Chaos fracturing in Unreal Engine

shape thus creating a pattern that is often seen in nature [6]. This process, as shown in figure 1, is both applicable to 3D and 2D shapes and can be combined with other techniques (such as Voronoi clipping) to fracture meshes, creating realistic destruction effects [52]. This technique is used in Unreal Engine's Chaos system [18] (shown in figure 2), the battlefield series [16] and many other scenarios [47] [36] [9].

2) *Polygon Clipping*: Polygon clipping is a method for destruction in games that involves using a polygon intersection algorithm like Greiner-Hormann to separate intersecting polygons into new shapes [21]. A triangulation algorithm is then used to regenerate the mesh in the cut shape. To create 3D geometry, the mesh can be extruded [31]. However, this is limited to relatively simple, planar surfaces, making it unsuitable for more complex 3D destruction. For complex meshes techniques such as Voronoi-based fracturing are preferred as they can handle more intricate and non-planar geometries. This

has been used in Rainbow Six: Siege's destruction of walls. [31]

3) *Voxel-Based Destruction*: Voxels are small volumetric units that typically take the shape of cubes [2]. Voxel-based destruction systems work by dividing objects into these voxels. They can then add, modify, or remove the voxels to simulate destruction. An example of a game based on this is Teardown [27]. The main restriction is that this offers relatively poor performance due to the number of voxels that must be processed in real time.

4) *Pre-computed Destruction*: Pre-computed destruction is a technique where the destruction is calculated before the program is run typically in the editor [48] with the results stored. These results are then swapped in during runtime when an impact is detected [41]. There are a variety of methods to achieve this, such as using Voronoi to pre-calculate the Fractured mesh as featured in Unreal Engine's Chaos system [18]. The benefits of this are its efficiency, as the expensive fracturing process is performed beforehand, and more artistic control as developers can influence the shape of fractured pieces. The disadvantage to this is the time it can take to design all the destruction assets used so it is often more efficient to use procedural destruction as this is performed automatically instead of manually. It can also be considered less realistic as it is done by interpretation instead of simulation. This has been used in games, such as Battlefield and its Frostbite engine [16]. It's also used in the films and TV industry with shows like The Last of Us, where handmade destroyed buildings were used to create a view of a destroyed Boston [15].

D. Voronoi Clipping

When computing a Voronoi fracture of a mesh, the bounding box of the mesh is often used for ease of computation. This, however, results in the generated fractures not matching the mesh and escaping its bounds. To fix this, the Voronoi diagram must be clipped to the mesh using Voronoi clipping [52]. This is called a clipped Voronoi diagram. It works by first computing all the boundary sites of the diagram against the mesh to get the Restricted Voronoi diagram (RVD) [14]. Once all boundary sites have been found, they can be clipped against the mesh. For each boundary point, the intersection of the cell and tetrahedron (tet) can be computed and clipped using the Sutherland-Hodgman algorithm [46]. During this process, it checks if the opposite cell on that plane is a boundary cell, and if it has been processed. If it has, a new pair has been found and it is added to the list "Q". The tets that share a face with the current tet are also checked and, if they have not been added to the incident tet set of the current cell, they are added. This process then ends when Q is empty and leaving a clipped mesh.

E. Triangulation

When rendering an object, it is often made up of many triangles. Each triangle is passed to the GPU to be rendered. However, this method often results in wasted space as points in a triangle can repeat. As a solution to this, index's are used

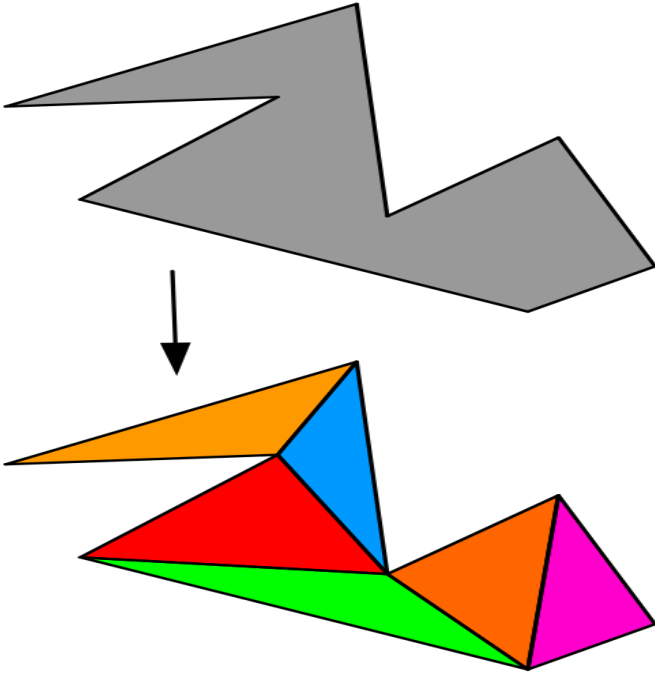


Fig. 3. An example of triangulation

to define the triangles from a set of points. However, when altering an object, the index order can be destroyed which results in the requirement that it is recalculated and rebuilt. This is where triangulation is used. Triangulation is the process of generating a set of indices from a set of points as shown in figure 3.

1) *Ear Cutting Triangulation*: Ear-cutting triangulation is an $O(n^2)$ triangulation algorithm that works by finding the ear of a polygon which is defined as three consecutive vertices, V_0, V_1, V_2 where V_1 is a convex vertex. If no other vertices are inside this, the ear acts as a triangle in the polygon. This then repeats removing V_0 from the list. There are more efficient algorithms available but they are more complex to implement.

2) *Delaunay Triangulation*: Delaunay triangulation was first invented by Boris Delaunay [12] and is the geometric dual of the Voronoi diagram [7]. Delaunay defined this as two points being connected when they are the only points in the interior of a circle. Delaunay triangulation has been used in early mesh generation [37] and is still used today due to the algorithm being highly optimised [7] with a time complexity of $O(n \log n)$.

3) *Constrained Delaunay Triangulation*: Constrained Delaunay Triangulation (CDT) [10] is similar to Delaunay triangulation but the pre-specified edges are included in the triangulation. This means the original shape of the mesh is preserved. This will be necessary for Voronoi diagram generation as the exact shape is needed and not just a hull from the points. CDT also has the same time complexity as Delaunay triangulation at $O(n \log n)$ [10]. However, as this paper will be using the algorithm in 3D, an adjusted version is needed that has a worst case time complexity of $O(n^2)$ [25].

F. Point Sampling

An important part of fracturing a mesh using Voronoi is generating the points used in the algorithm. More evenly spread points can create more realistic destruction effects [53] and avoids points being too close to each other, which can lead to floating point errors.

1) *Random Sampling*: Random sampling is a fast method of generating points but can result in points being the same (or very close) and results in very irregular cell shapes being formed. It consists of using a pseudo-random number generator, such as the Mersenne twister [33], to generate a point inside a given boundary.

2) *Maximal Poisson-Disk Sampling*: Maximal Poisson-disk sampling (MPS) selects random points in a boundary with an exclusion/inclusion radius r that ensures no two points are closer than r to each other [13]. This works well with Voronoi leading to good quality cells as it is bound to the aspect ratio of the cell [13] [26].

III. RESEARCH QUESTIONS

A. Conclusions from Background

1) *API*: The background suggests that Vulkan will perform better than OpenGL on modern GPU's due to it being designed with those in mind [8] and having more control than OpenGL [23]. However, this could be change on older GPU's where OpenGL was designed for use and hardware optimised against.

2) *Triangulation*: This experiment will be using constrained Delaunay triangulation, due to its speed over ear clipping. This will be necessary as performance is being measured, so a highly optimised algorithm would be ideal [7]. Constrained Delaunay triangulation has been chosen over Delaunay triangulation due to its ability to retain the shape of the polygon unlike Delaunay which creates a hull from the points with no regard to the original shape.

3) *Sampling*: For sampling, Poisson sampling is the ideal option, being optimised, whilst keeping a set distance between points [13] reducing the floating point errors that random sampling is at risk of. Due to this, Poisson sampling is ideal and used in this paper.

4) *Other Research*: This paper could not find other research that explicitly compares the performance of Voronoi cell generation on different APIs, such as OpenGL and Vulkan. This suggests there is a research gap in this area.

5) *Rational*: From this, this paper can conclude that there is a real need for this research as Voronoi is often used for destruction in applications [11]. However, these algorithms are computationally expensive, and contemporary literature constantly highlights the need for performance in making such algorithms practical for applications such as game engines [34] [20] [1]. Whilst literature exists on the optimization of the algorithms themselves [52] [14] [7], there is a notable lack of work on how the choice of graphics API effects the performance of such algorithms. The graphics API is important to the performance of these algorithms as different APIs have varying effects on the performance of the GPU which will

have a direct effect on the processing speed of the algorithms if implemented on the GPU. However, if implemented on the CPU it will effect the transfer speed of data from the CPU to GPU, changing the time to render [30] [8] [23]. Thus, producing clear results on the performance benefits of each API would prove useful to those looking to optimise an application for destruction using Voronoi fracturing.

B. Hypothesis

After the background the research question can be concluded as: *How does OpenGL compare to Vulkan in performance for the 3D Voronoi fracturing of a complex mesh?* The derived hypotheses are:

Hypothesis	Null Hypothesis
OpenGL will perform slower than Vulkan at Voronoi cell generation on modern GPUs	OpenGL will be the same speed as Vulkan at Voronoi cell generation on Modern GPUs
OpenGL will perform slower than Vulkan at Voronoi clipping on modern GPUs	OpenGL will be the same speed as Vulkan at Voronoi clipping on Modern GPUs

IV. METHODOLOGY

A. Philosophy

This research takes a pragmatic [32] approach to the ontological position as this paper is viewing a single reality that is the performance of current hardware for current Voronoi fracturing algorithms. However, this reality is subject to change with the social aspect of advancements in technology, be it in the hardware or algorithms that will require this research to be re-evaluated. The epistemological position this paper takes is that of positivism as the metric this paper is measuring, performance time, is Objective, observable and empirical. This means it can be measured and quantified providing robust results.

B. Gathering Data

To acquire the data, a G*Power-calculated sample size of 145 [4] points will be used for each point-set of size 10, 100 and 1000. These point sets will be unique, generated once and stored in a json file. This is to ensure that each machine the artefact is tested on has the same points preventing bias if a set of points proves to be faster than the other. Multiple sets are being tested on to measure whether Vulkan or OpenGL performs better at lower scales and the test repeated 145 times each to ensure enough data is collected to avoid outliers. The timer to measure performance will start when the Voronoi diagram is first being created. The program will have already loaded the first set of points. This is because the focus is on the generation of a Voronoi diagram not loading data from a file, which would bias the data if added. The program will then go through each set generating a Voronoi diagram and clipping it to the mesh. After going through all 145 points of this set, it will pause the timer, calculate the mean performance time, and record it to a json file (shown in figure 1). The timer will then reset and the next set will be loaded before repeating this

process. This will continue until all sets have been completed. Then, the program will switch to the next API and repeat the process again, saving the data in a different column. The time will be measured in milliseconds(ms) as on the lower point sets a measurement less granular would show small deviations and become difficult to compare. After the completion of the test sets, the hardware specifications of the computer used to run the tests will be recorded allowing performance on different hardware to be compared.

```

1  [
2    {
3      "Data": {
4        "Card": "testcard",
5        "OS": "testOS",
6        "RAM": "TestRam",
7        "CPU": "TestAPI",
8        "API": "TestAPI",
9        "10": [
10         "Test10",
11         "Test10"
12       ],
13       "100": [
14         "Test100",
15         "Test100"
16       ],
17       "1000": [
18         "Test1000",
19         "Test1000"
20       ]
21     },
22   ],
23 ]

```

Listing 1. Json test data example format.

C. Variables

1) *Hardware*: To answer the research question, it is pivotal that the artefact is tested on a variety of hardware, thus the need to gather the hardware data for comparison. Unfortunately, due to resource limitation, tests will only be able to be conducted on a few computers. The hardware data being gathered is the graphics card model, operating system, Ram size and CPU. These are all necessary as they could effect the outcome of the program.

2) *Number of points*: To provide a more realistic fracture effect, it is important there are enough points to generate a Voronoi diagram. However, with more points generation of both the diagram and clipping will take longer. This could also effect the performance of the APIs, as Vulkan, with its focus on parallelization, is predicted to perform better with more points but OpenGL could perform better with a lower number of points. To ensure reliability of the results, tests are going to be committed on a varying number of points, 10, 100 and 1000.

3) *Mesh*: Another important factor that could effect performance is the mesh used in the clipping stage. Although this won't have an effect during point generation, the number of triangles a mesh has will effect how long clipping takes. This could also effect the APIs as again Vulkan is predicted to perform better with more data and OpenGL could perform better with less. To mitigate the risk of this variable effecting

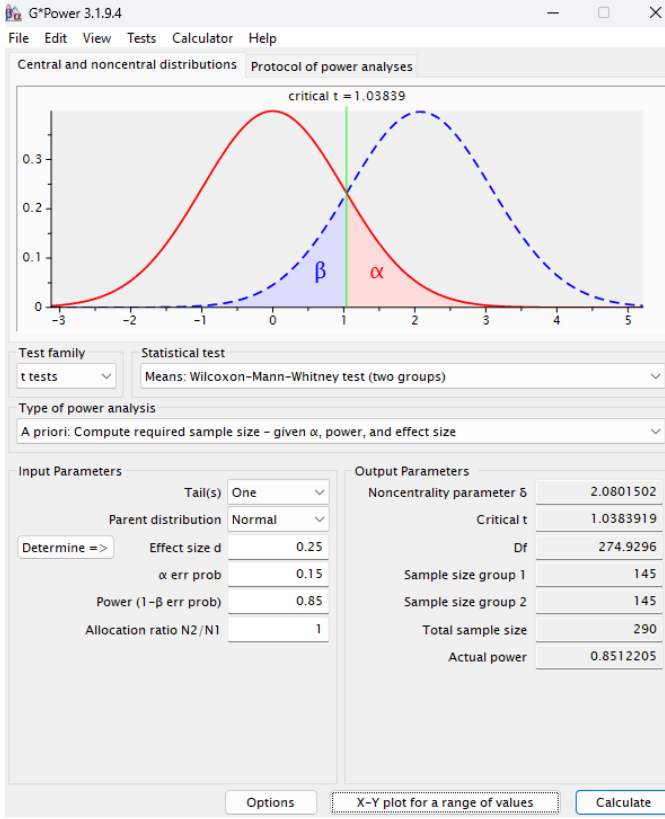


Fig. 4. G*Power test

the results, testing will be done using one mesh on all machines.

D. Testing

This research involves one dependent variable, the performance time in milliseconds, and one independent variable with two levels (API: Vulkan vs OpenGL). The performance time is considered interval and is not assumed to be normally distributed. As the performance time often exhibits skew and the groups are independent, a Wilcoxon-Mann Whitney test [44] will be used to compare Vulkan and OpenGL for each set of the data. As shown in figure 4, G*Power calculated sample size needs to be 145.

Each dataset gathered per hardware configuration will be plotted using a box plot as this would clearly show the distribution of execution time and highlight any outliers in the data. The Wilcoxon-Mann Whitney test will be performed using the library "scipy.stats", which contains a mannwhitneyu function.

E. Summary

These methods answer the research question as the performance of these algorithms is being measured on different APIs using different point sets of varying sizes, ensuring the scalability of the algorithms is sufficiently tested. This is then repeated 145 times using different point sets to ensure results are reliable. This paper can then conclude the performance

difference between the two APIs, answering the question of how the two APIs compare in performance for 3D fracturing of a complex mesh.

F. Quality assurance

To ensure the quality of the code unit tests will be conducted using polarity profiling [17]. This will test the code with tests that are expected for the system to be able to handle and unexpected tests that the system may not be build to solve.

To ensure code readability, the cyclomatic complexity, maintainability index, and lines of code will also be measured. These measures are not absolute, but will give a general idea of the readability of the program.

Where appropriate, assert statements will also be used to assure that all parts of the program are producing the expected results.

This ensures an accurate view of the systems capabilities are given, with its constraints obvious.

V. COMPUTING ARTEFACT

A. Overview

The artefact will be a basic 3D game engine allowing for basic rendering of a 3D object, camera movement and the ability to explode the fractured object. The engine will be using the bridge pattern to allow the easy swapping of graphics APIs as demonstrated in figure 3. This will allow the code for the two APIs to be produced to be as analogous as possible, reducing bias in the results that could result from different implementations. However, this results in a loss of the control that Vulkan offers as the computing artefact is limited to systems that both APIs can access. Additionally, the engine will include a timer to measure the performance of the Voronoi diagram generation, clipping and triangulation classes that will be tested. A class outline of the engine can be found at figure 6. Table I highlights the libraries used in the source code.

B. Alternatives

Voro++ [40] is a library that implements Voronoi tessellation. It was considered as the main algorithm used to generate Voronoi diagrams but use was decided against in this test due to it's inability to perform Voronoi clipping against a mesh. CDT [4], a library for constrained Delaunay triangulation was considered but was also ruled out due to being restricted to 2D, whereas this test is being conducted in 3D.

C. Runtime Engine and subsystems

At the core of the program is the runtime engine class. As shown in figure 3, this is the factory class used to initialise different APIs by changing the namespace before creation. This will handle all engine specific code, that being initialisation, the core loop, shutdown and clean-up of all subsystems. For faster development the engine contains an SInstance class, which is a singleton, designed to allow the engine to quickly access key resources but adds dependence on its existence, reducing scalability and flexibility. The engine handles the creation of all subsystems which can be seen in figure II.

All subsystems have been enclosed in a try catch block for error trapping and handling printing what went wrong if any fail. The Loop function calls all the registered tick functions from objects and then the rendering loop, to ensure rendering isn't attempted before all objects have completed their actions this frame. The shutdown function handles the cleanup of all subsystems ensuring everything is shutdown without resource leaks.

D. Objects

To allow any class to access the rendering loop and have its own tick function that is called every frame, an object class has been implemented. The object class has a Start and Tick function so any class that inherits from it will have their own loops where custom code can be added. However, an object needs to be created using the "ObjectFactories" "CreateObjectPtr" or "CreateObjectRef" functions. This will trigger the start function which registers the tick function with the static multicast delegate TickDel. This is shown in examples 4, 5 and 6. This is then called by the runtime engine. It is beneficial as it allows easy access to a loop function but comes with less scalability as more objects will increase the burden each frame and increase memory usage due to many functions being registered to TickDel.

E. Bridge Objects

In the engine there are resources that will be used outside of the Runtime engine but have differing API implementations such as shaders. To solve this, the bridge pattern [38] has been used, providing an interface class for the different APIs. This involves creating four different classes, Interface, Base, Vulkan and OpenGL. The interface owns a static function pointer to a function that returns a Base. The creation function belongs to either Vulkan or OpenGL and will create the resources in the way needed. This is loaded during Engine initialisation. The interface will then own the returned Base and when any function is called, it will call the corresponding function on Base. This is shown in examples 7, 8, 9. This ensures API specific code is encapsulated and internal to the specific API class. This means when switching engines, Vulkan or OpenGL specific features can be taken advantage of for efficiency increasing the amount of control. It also increases the portability of the system, as if the GPU doesn't support one type of API, the other can be attempted. However, it reduces the scalability and flexibility of the system as anything called by the interface has to be implemented by both OpenGL and Vulkan or translated by the implementing class and the creation process of classes that requires API code is now longer, extending development time.

F. Rendering

The render has a bridge class that wraps adding items to be rendered. However, unlike the normal bridge pattern this doesn't have a creation function but has the rendering instance directly set by the engine. This is because only one rendering class is in use at a time. The render class can only render

certain classes, for example, the FracturePiece3D or Model classes. This means specific functions are needed to add each to the renderer, which increases scale and reduces maintainability. Implementing a base class that rendered objects can inherit from and adding them to a collection such as a list would be an alternative method that would allow for better scalability, maintainability and flexibility. However, this was discarded as it would have made removing specific groups, such as debugging shapes, from the renderer more complex, making debugging harder and increasing development time.

G. Voronoi Diagram Generation

The Voronoi diagram is created in the Voronoi class. This will generate the FracturePiece3D's, and store them in the Voronoi class. This class can then be passed into the clipping class to clip the diagram to a mesh.

1) *Triangulation*: This calls the triangulation class's Triangulate function on each fracture piece, to ensure each piece is indexed properly as otherwise it can cause visual bugs. The triangulate function takes an array reference to the vertices and an array reference to the indices which will also function as the output.

H. Voronoi Clipping

The Voronoi clipping is a separate class that takes a Model and an array of FracturePiece3D's. This clips the mesh by the fracture pieces, returning an array of Meshes, which are rendered as the final clipped model.

VI. VALIDATION AND TESTING

A. Unit tests

To ensure the artefact is producing the correct results and all systems are working as intended, unit tests have been built. These run part of the program and compare the result against an expected result. If they match, the test passes, if not, it fails and flags that test and part of the program as having an error. This ensures the stability of the results, contributing to the maintainability of the program. These unit tests are implemented for core aspects of the artifact such as the Array class as shown in figure 10.

B. User Expectations Testing

In order to test that the result of diagram generation and fracturing matches what is expected, User Expectations Testing (UET) is going to be carried out. This involves looking at the visual results of the program and checking if it matches what is expected. For Voronoi, this involves ensuring the diagram is generated correctly and the mesh appears to have been fractured appropriately. If not, then there could be a problem with the generation. This will be used in combination with unit testing and focused on the visual effects of the artefact, helping to ensure the artefact is of a suitable quality.

C. Artefact design

To reduce any bias in the results due to differing implementations a bridge based interface between the different graphics APIs has been implemented. This separates the Graphics API implementations and the Voronoi implementation, ensuring the results are specific to the API and are not effected by differing Voronoi Implementations. This is beneficial for maintainability as instead of implementing the same code twice for two different APIs, it only needs to be implemented once. In addition, this design significantly improves maintainability as the Voronoi logic is implemented only once, while each graphics API provides its own implementation reducing duplication and simplifying future extensions.

D. Shaders

The artefact does not need complex shaders as the mesh fragmentation will be only rendering a mesh and texture. All shader code is implemented in GLSL to maintain a single consistent implementation across both graphics APIs. However, Vulkan requires the shader format to be SPIR-V, which can be compiled from GLSL code, provided it is formatted correctly. This ensures the same shader code is being used and that any differences in performance are from the APIs rather than shader code.

E. Measurement Code

To record the performance data, this paper has opted to use a library for writing to a json file, nlohmann json [29]. As shown in figure 11, the data will be saved using the DataRecorder class, saving a record, which contains the GPU, OS, RAM, CPU, API and the execution time for each point set. Using a single function that takes a struct improves maintainability and scalability, allowing additional fields to be easily added as required. Because this function is implemented as an exposed static class member function it is easily accessed, ensuring high usability.

VII. DATA MANAGEMENT

A. Collecting Data

To provide accurate results, it is important to test the algorithm on a wide variety of graphics cards. However, an insufficient amount were available for this experiment, so testing will only occur on a few different machines. Data will be collected by first closing all other applications, then the application will be run, which will automatically run all tests and output the data to a json file. This output will then be uploaded to the One-Drive and git. The data collected will be the type of graphics card, operating system, ram size, CPU, API, the processing time taken and how many points were generated. None of this data is personal information and so doesn't need to follow any of the regulation set in GDPR [49]. However, this data is still important so it will be stored locally on a machine and uploaded to both a git server and One-Drive as backup.

B. Reading The Data Using Pandas

To read the saved json performance data, this paper uses the `pandas.read_json` function. The data is then used to generate a box plot, where the x-axis represents the graphics API and the y-axis represents the execution time. A box plot was selected because frame-time measurements frequently contain spikes or other outliers. Box plots reveal these outliers without distorting the underlying distribution, allowing for a clear and fair comparison between the two APIs. This process is shown in figure 12. The results from the Wilcoxon-Mann Whitney test will be stored in a text file, stating set tested, the statistic, and the p value. This will allow the results to be clearly seen. An example of this can be seen in figure 2

```
1 Set: 10 ,statistic: 126.0 , p_value:
   0.532690480327402
2 Set: 10 ,statistic: 126.0 , p_value:
   0.532690480327402
3 Set: 100 ,statistic: 8.0 , p_value: 1.0
```

Listing 2. An Example of how test results are layed out.

VIII. CONSIDERATIONS

A. Legal

The Voronoi diagram, as well as Voronoi clipping, are in the public domain and not restricted by any license so this paper is free to use, distribute and publish results based on it. None of the data collected is personal information so it doesn't have to adhere to any GDPR [49] rules. OpenGL is licensed under the OpenGL Architecture Review Board (ARB) license. This permits anyone to use, reproduce and distribute the software freely. Vulkan is also open source and thus free for use. It is important to clarify that whilst the graphical APIs are free to use, the drivers that define the implementation of these APIs are not. However, as this paper does not modify any of these drivers, it is not an issue.

B. Social

1) *Performance*: This paper aims inform developers about which graphics API is appropriate when including destruction techniques in games with the aim of maximizing performance. This would have clear social impacts as better optimized games can run better on lower end hardware, increasing accessibility for users who cannot access higher end hardware. By reducing hardware barriers, this paper makes applications that utilize Voronoi more accessible and inclusive for a wider demographic.

2) *API Shift*: If one API shows a significant performance advantage over the other it may influence future development trends. A shift towards either Vulkan or OpenGL could result in increased hiring for those APIs and training priorities to shift towards that API. As the industry adapts to the more efficient API for Voronoi based destruction, so to will educational criteria and hiring practices need to change.

C. Ethical

This research is low risk as it does not involve any participants or personal data and as such does not violate GDPR [49]. It does not access any sensitive data, or have a significant environmental impact. It does not have a potentially harmful impact. It does not meet any of the other ethical risks on the ethical risk checklist either. This project therefore does not violate the nuremburg code [28], the BCS Code of practice [43], or the research philosophy. As a result of this there aren't many ethical considerations to make for this project. This project has gone through the ethical review process and has been approved as low risk 7.

D. Professional

It is important this paper adheres to the BCS code of conduct [43], which states that one should not claim a level of competence that one does not possess. This paper does this by making it clear results have not been fully tested on a wide verity of machines. The code of conduct, along with the Nuremberg code [28], also states that the research conducted should be for the betterment for society, which this paper achieves by providing research that can only benefit the profession.

E. Sustainability

Games as a whole can take a large of electricity [35] to use. Therefore, this project will benefit the sustainability of the games industry because its focused on researching what API can run destruction algorithms faster, which as a by-product, would also serve to reduce electricity costs, making destruction algorithms as a whole more sustainable.

IX. RESULTS

X. ANALYSIS

XI. CONCLUSION

A. Future work

XII. PROJECT PLAN

The further development of the computing artefact will follow the waterfall methodology. This approach is appropriate because the project has predefined features that background research has shown is unlikely to change. Because of this, the focus is on implementing the pre-planned features in a structured sequence. Waterfall supports this style of development, as each stage is completed before the next begins. For example, the process will move from implementing the OpenGL renderer, to generating Voronoi diagrams and finally to producing Delaunay triangulations. The full development schedule is shown in Figure 8. The major milestones this project expects are the ethics approval, Proposal Assessment session, Completion of the computing artefact, Drawing a conclusion and Assignment hand in. The gantt chart 8 details the exact dates this project expects to finish these milestones. The ethics approval has been completed and approval secured. The project is currently on schedule and is expected to finish all tasks on time. This project will be monitoring the progress

against the gantt chart to adjust if any part is falling behind, or ahead of schedule. In the case of falling behind, parts that are more ahead will be put on hold whilst this part is being finished. In the case of finishing early, the next step will be brought forward and in the case of finishing all steps early additional time may be spent on Optimization and refinement. In the case of the project being unlikely to finish by the deadline, lower priority parts of the project may be cut, for example the shifting of algorithms to a compute shader. This will ensure the project finishes in a timely manner.

XIII. GITHUB

Repository can be found at
<https://github.falmouth.ac.uk/GA-Undergrad-Student-Work-25-26/Dissertation-SL295211.git>

REFERENCES

- [1] Tomas Akenine-Möller, Eric Haines, Naty Hoffman, Angelo Pesce, Michał Iwanicki, and Sébastien Hillaire. *Real-time rendering*. A Balkema book. CRC Press, Boca Raton London New York, fourth edition, first issued in hardback edition, 2019.
- [2] Mitko Aleksandrov, Sisi Zlatanova, and David J. Heslop. Voxelisation algorithms and data structures: A review. *Sensors*, 21(24):8241, January 2021.
- [3] AMD. AMD Vulkan™ Graphics API. Published at <https://www.amd.com/en/products/graphics/ecosystems/vulkan.html>.
- [4] Artem Amirkhanov. artem-ogre/cdt, November 2025. Published at <https://github.com/artem-ogre/CDT>.
- [5] Apple. Metal Overview. Published at <https://developer.apple.com/metal/>.
- [6] Franz Aurenhammer. Voronoi diagrams—a survey of a fundamental geometric data structure. *ACM Comput. Surv.*, 23(3):345–405, September 1991.
- [7] Franz Aurenhammer, Rolf Klein, and Der-tsay Lee. *Voronoi Diagrams And Delaunay Triangulations*. World Scientific Publishing Company, June 2013. Google-Books-ID: cic8DQAAQBAJ.
- [8] Axel Blackert. *Evaluation of Multi-Threading in Vulkan*. DiVA, 2016. Published at <https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-132025>.
- [9] Martin Bock, Amit Kumar Tyagi, Jan-Ulrich Kreft, and Wolfgang Alt. Generalized voronoi tessellation as a model of two-dimensional cell tissue dynamics. *Bulletin of Mathematical Biology*, 72(7):1696–1731, October 2010.
- [10] L. P. Chew. Constrained delaunay triangulations. In *Proceedings of the Third Annual Symposium on Computational Geometry*, SCG '87, page 215–222, New York, NY, USA, 1987. Association for Computing Machinery.
- [11] Matthew Clothier and Mike Bailey. Creating destructible objects using a 3d voronoi subdivision tree. In *2015 15th International Conference on Computational Science and Its Applications*, pages 42–46, 2015.
- [12] B. Delaunay. B.delaunay, sur la sphere vide. a la memoire de georges voronoi, izv. math., 1934, no.6. Published at https://www.mathnet.ru/php/archive.phtml?wshow=paper&jrnid=im&paperid=4937&option_lang=eng#forwardlinks.
- [13] Mohamed S. Ebeida and Scott A. Mitchell. Uniform random voronoi meshes. In William Roshan Quadros, editor, *Proceedings of the 20th International Meshing Roundtable*, pages 273–290, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg.
- [14] Herbert Edelsbrunner and Nimish R. Shah. Triangulating topological spaces. In *Proceedings of the tenth annual symposium on Computational geometry*, SCG '94, page 285–292, New York, NY, USA, 1994. Association for Computing Machinery. Published at <https://dl.acm.org/doi/10.1145/177424.178010>.
- [15] Ian Failes. Dneg made a procedural tool that let artists create bomb-shaped holes in boston buildings for 'the last of us', March 2023. Published at <https://beforeandafters.com/2023/03/18/dneg-made-a-procedural-tool-that-let-artists-create-bomb-shaped-holes-in-boston-buildings-for-the-last-of-us/>.
- [16] Battlefield fandom. Battlefield wiki on destruction, October 2025. Published at <https://battlefield.fandom.com/wiki/Destruction>.

- [17] Norman Fenton. Software quality: Theory and management. alan c. gillies. published by chapman & hall, london, u.k., 1992. isbn 0 412 4513 0, 250 pages. price: £19.95, soft cover. *Software Testing, Verification and Reliability*, 2(3):155–155, sept 1992.
- [18] Epic Games. Chaos destruction overview. Published at https://dev.epicgames.com/documentation/en-us/unreal-engine/chaos-destruction-overview?application_version=4.27.
- [19] Epic Games. Graphics programming overview. <https://dev.epicgames.com/documentation/en-us/unreal-engine/graphics-programming-overview-for-unreal-engine>.
- [20] Jason Gregory. *Game Engine Architecture*. A K Peters/CRC Press, New York, 3 edition, July 2018.
- [21] Günther Greiner and Kai Hormann. Efficient clipping of arbitrary polygons. *ACM Trans. Graph.*, 17(2):71–83, April 1998.
- [22] Khronos group. OpenGL - The Industry Standard for High Performance Graphics. Published at <https://www.opengl.org/>.
- [23] Khronos Group. What is Vulkan? :: Vulkan Documentation Project. Published at https://docs.vulkan.org/guide/latest/what_is_vulkan.html.
- [24] Knronos Group. Vulkan. Published at <https://www.vulkan.org/>.
- [25] Barry Joe. Construction of three-dimensional delaunay triangulations using local transformations. *Computer Aided Geometric Design*, 8(2):123–142, 1991.
- [26] Patrick M. Knupp. Algebraic mesh quality metrics. *SIAM Journal on Scientific Computing*, 23(1):193–218, 2001. Published at <https://doi.org/10.1137/S1064827500371499>.
- [27] Tuxedo Labs. Teardown, voxel game technical q&a deep dive (twitch archive). Published at <https://archive.org/details/teardown-voxel-game-technical-q-a-deep-dive-twitch-archive>.
- [28] Andrew Ivy Leo Alexander, Harold Sebring. Nuremberg Code. Published at https://research.unc.edu/human-research-ethics/resources/ccm3_019064/.
- [29] Niels Lohmann. nlohmann/json: Json for modern c++. Published at <https://github.com/nlohmann/json>.
- [30] Michael Lujan, Michael Baum, Dayuan Chen, and Ziliang Zong. Evaluating the performance and energy efficiency of opengl and vulkan on a graphics rendering server. In *2019 International Conference on Computing, Networking and Communications (ICNC)*, pages 777–781, 2019.
- [31] Julien L’Heureux. Art of destruction in ‘rainbow six. Published at <https://gdcvault.com/play/1023003/The-Art-of-Destruction-in->
- [32] Heba Maarouf. Pragmatism as a supportive paradigm for the mixed research approach: Conceptualizing the ontological, epistemological, and axiological stances of pragmatism. *International Business Research*, 12:1–1, 08 2019.
- [33] Makoto Matsumoto and Takuji Nishimura. Mersenne twister: a 623-dimensionally equidistributed uniform pseudo-random number generator. *ACM Trans. Model. Comput. Simul.*, 8(1):3–30, January 1998. Published at <https://doi.org/10.1145/272991.272995>.
- [34] Farouk Messaoudi, Adlen Ksentini, Gwendal Simon, and Philippe Bertin. Performance analysis of game engines on mobile and fixed devices. *ACM Trans. Multimedia Comput. Commun. Appl.*, 13(4), September 2017.
- [35] Evan Mills, Norman Bourassa, Leo Rainer, Jimmy Mai, Arman Shehabi, and Nathaniel Mills. Toward Greener Gaming: Estimating National Energy Use and Energy Efficiency Potential. *The Computer Games Journal*, 8(3):157–178, December 2019. Published at <https://doi.org/10.1007/s40869-019-00084-2>.
- [36] P. A. Mulheran and J. A. Blackman. Capture zones and scaling in homogeneous thin-film growth. *Phys. Rev. B*, 53:10261–10267, Apr 1996.
- [37] Oleg R. Musin. Properties of the delaunay triangulation, 2003. Published at <https://arxiv.org/abs/math/0312050>.
- [38] Dmitri Nesteruk. *Design patterns in modern C++: reusable approaches for object-oriented software design*. Apress, Place of publication not identified, 2018.
- [39] Atsuyuki Okabe, Barry Boots, Kokichi Sugihara, Sung Nok Chiu, and D G Kendall. Spatial tessellations: Concepts and applications of voronoi diagrams, second edition. *Wiley Series in Probability and Mathematical Statistics*, Chichester, New York: Wiley, 1992, 1992.
- [40] Chris H. Rycroft. Voro++ - a 3d voronoi cell software library. Published at <https://math.lbl.gov/voro++/>.
- [41] Silvia Sellán, Jack Luong, Leticia Mattos Da Silva, Aravind Ramakrishnan, Yuchuan Yang, and Alec Jacobson. Breaking good: Fracture modes for realtime destruction. *ACM Trans. Graph.*, 42(1):10:1–10:12, March 2023.
- [42] Peter Shirley, Michael Ashikhmin, and Steve Marschner. *Fundamentals of Computer Graphics*. A K Peters/CRC Press, New York, 3 edition, July 2009.
- [43] British Computer Society. BCS Code of Conduct for members - Ethics for IT professionals | BCS. Published at <https://www.bcs.org/membership-and-registrations/become-a-member/bcs-code-of-conduct/>.
- [44] SPSS Stata and R. Choosing the correct statistical test in sas. Published at <https://stats.oarc.ucla.edu/other/mult-pkg/whatstat/>.
- [45] stevewhims. DirectX graphics and gaming - Win32 apps. Published at <https://learn.microsoft.com/en-us/windows/win32/directx>.
- [46] Ivan E. Sutherland and Gary W. Hodgman. Reentrant polygon clipping. *Commun. ACM*, 17(1):32–42, January 1974.
- [47] Daniel Sánchez-Gutiérrez, Melda Tozluoglu, Joseph D Barry, Alberto Pascual, Yanlan Mao, and Luis M Escudero. Fundamental physical cellular constraints drive self-organization of tissues. *The EMBO Journal*, 35(1):77–88, January 2016.
- [48] Rachel Thomas and Wenshu Zhang. Real-time fracturing in video games. *Multimedia Tools and Applications*, 82(3):4709–4734, January 2023.
- [49] European Union. General data protection regulation (gdpr) – legal text. Published at <https://gdpr-info.eu/>.
- [50] Unity. configure-graphicsapis. Published at <https://docs.unity3d.com/6000.0/Documentation/Manual/configure-graphicsAPIs.html>.
- [51] Georges Voronoi. New applications of continuous parameters to the theory of quadratic forms. *Z. Reine Angew. Math*, 134:198, 1908.
- [52] Dong-Ming Yan, Wenping Wang, Bruno Lévy, and Yang Liu. Efficient computation of clipped voronoi diagram for mesh generation. *Computer-Aided Design*, 45(4):843–852, April 2013.
- [53] Shen Ying, Guang Xu, Chengpeng Li, and Zhengyuan Mao. Point cluster analysis using a 3d voronoi diagram with applications in point cloud segmentation. *ISPRS International Journal of Geo-Information*, 4(3):1480–1499, 2015. Published at <https://www.mdpi.com/2220-9964/4/3/1480>.

APPENDIX A SOFTWARE STRUCTURE

```

1 Vulkan::RuntimeEngine engine;
2 engine.Init();
3
4 while (!engine.ShouldClose())
5 {
6     engine.Loop();
7 }
8
9 engine.ShutDown();
10
11 OpenGL::RuntimeEngine openGLEngine;
12 openGLEngine.Init();
13
14 while (!openGLEngine.ShouldClose())
15 {
16     openGLEngine.Loop();
17 }
18
19 openGLEngine.ShutDown();

```

Listing 3. Create an API Engine

```

1 /**
2  * The Base class of the engine, anything that needs
3  * a tick or start should inherit from this
4  */
5 class Object
6 {
7 public:
8     virtual ~Object() = default;
9
10 /**
11  * Called every frame

```

```

12  * @param DeltaTime The time since last frame
13  */
14  virtual void Tick(const double& DeltaTime);
15
16  /**
17   * All objects tick functions, the main game loop
18   * broadcasts this
19   */
20  static MulticastDelegate<const double&> TickDel;
21
22  /**
23   * Called on object creation, only once. Child
24   * classes should Implement the base version
25   */
26  virtual void Start();
27 };
28
29 void Object::Start()
30 {
31     TickDel.BindMember(this, &Object::Tick);
32 }

```

Listing 4. Object class and creation

```

1  class Camera final : public Object
2  {
3  public:
4      Camera(Window* Window, InputManager* Manager);
5
6      void Start() override;
7
8      void Tick(const double& DeltaTime) override;
9  }
10
11 camera = CreateObjectPtr<Camera>(Window,
    inputManager);

```

Listing 5. Object Instance creation Example

```

1  /**
2   * Creates a pointer to an object, allocated using
3   * new, so make sure to delete, or assign to a
4   * smart ptr
5   * @tparam T The Type of object to create
6   * @param args Any args the constructor of the
7   * object takes
8   */
9  template<typename T, typename... Args>
10 T* CreateObjectPtr(Args... args) requires obj<T>
11 {
12     T* object = new T(args...);
13     object->Start();
14     return object;
15 }
16
17 /**
18  * Creates an object of the specified type,
19  * allocated on the stack, so no need for
20  * management
21  * @tparam T The Type of object to create
22  * @param args Any args the constructor of the
23  * object takes
24  */
25 template<typename T, typename... Args>
26 T CreateObjectRaw(Args... args) requires obj<T>
27 {
28     T object = T(args...);
29     object.Start();
30     return object;
31 }

```

Listing 6. ObjectFactory creation functions

```

1  class ObjectInterface final
2  {
3  public:
4      ObjectInterface();
5
6      BaseObject* Base;
7      static std::function<BaseObject* ()>
8      ObjectCreationFunc;
9
10     void SomeFunc();
11 }
12
13 ObjectInterface::ObjectInterface()
14 {
15     Base = ObjectCreationFunc();
16 }
17 void ObjectInterface::SomeFunc()
18 {
19     Base->SomeFunc();
20 }

```

Listing 7. Interface Example

```

1  class ObjectBase
2  {
3  public:
4      virtual void SomeFunc() = 0;
5  }

```

Listing 8. Base Example

```

1  class ObjectVulkan : ObjectBase
2  {
3  public:
4
5      ObjectBase* CreationFunc();
6
7      void SomeFunc() override;
8  }
9
10 class ObjectOpenGL : ObjectBase
11 {
12 public:
13
14     ObjectBase* CreationFunc();
15
16     void SomeFunc() override;
17 }

```

Listing 9. Vulakn and OpenGL Example

Library	Version	Purpose
GLFW3	3.4	Window Creation
GLAD	0.1.36	OpenGL handler
Vulkan	1.4.328.1	Vulkan handler
glm	1.0.1	Maths Library
stb	2.3	Texture Loader
assimp	6.0.2	Mesh loader
nlohmann json	3.12	json writer

TABLE I
LIBRARIES USED IN SOURCE CODE

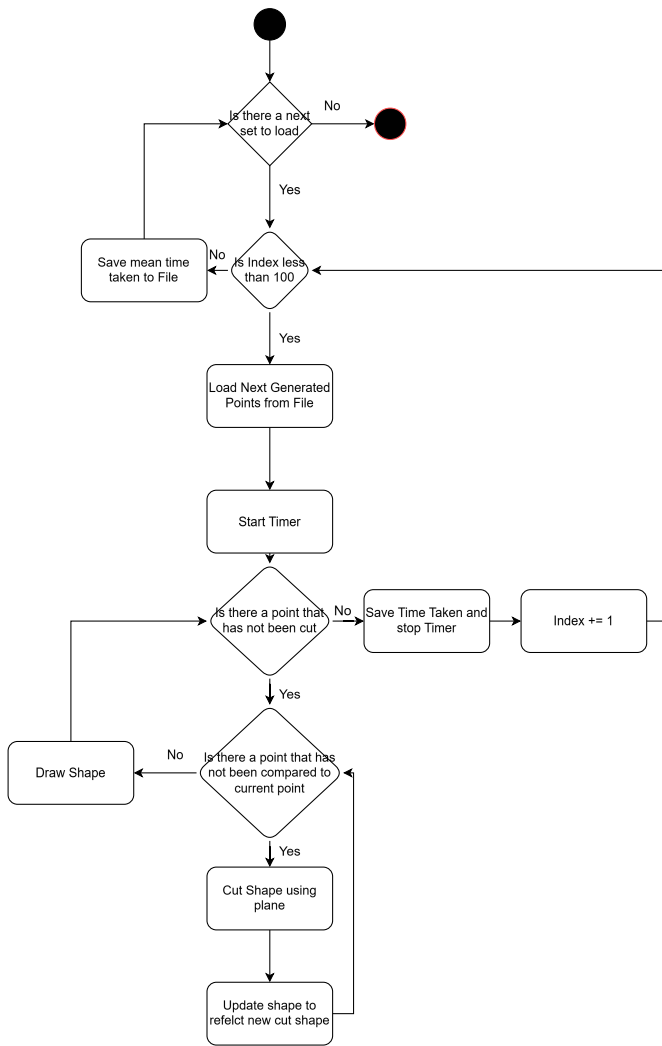


Fig. 5. A flowchart showing how voronoi diagrams are created and timed

SubSystem	Purpose	Specific To
Path Manager	Finds all needed paths local to the machine the artefact is loaded on, I.E. the path to the contents folder, and stores them in static variables	Shared
Initialize Vulkan	Initializes the Vulkan instance	Vulkan
Initialize OpenGL	Initializes the OpenGL instance	OpenGL
FirstWindow	Creates the window, and attaches the API to it. There are different implementations of this class for Vulkan and OpenGL, but they both inherit from window, with shared functions and variables stored there	Shared
BridgeManager	Sets the creation functions of the interface classes, to insure they use the created API code	Shared
UGraphicsCard	Responsible for the creation of the renderer, and Vulkan specific resources like the swap chain and logical device	Vulkan
OpenGLRenderer	Handles the rendering loop	OpenGL
Input Manager	Handles all inputs	Shared
World	Responsible for the static variables like deltaTime	Shared
Camera	Responsible for calculating the View and projection matrices, and handles movement	Shared

TABLE II
SUBSYSTEMS IN THE SOURCE CODE

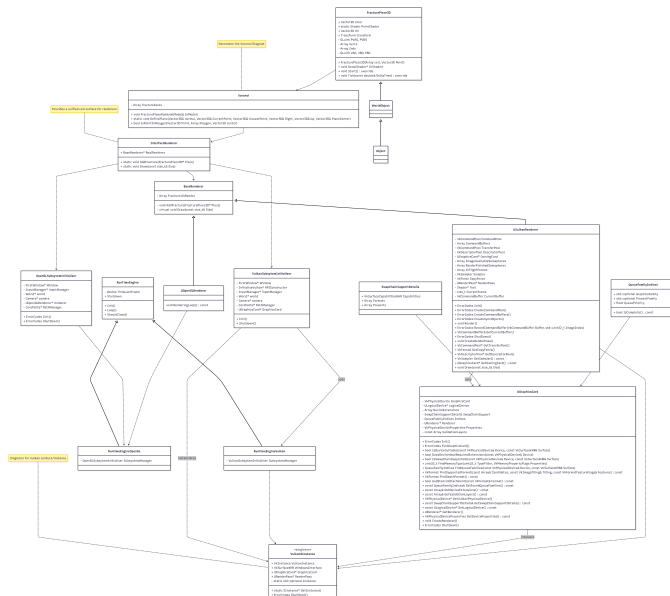


Fig. 6. A Class Diagram showing the creation of a voronoi Diagram

APPENDIX B ARTEFACT TESTING

```

1 TEST(Array, Copying)
2 {
3     Array<int> test;
4
5     test.Add(5);
6
7     test.Add(3);
8
9     Array<int> copy = test;
10
11     bool btest = copy == test;
12
13     ASSERT_EQ(btest, true);
14
15     copy.Remove(3);
16     btest = copy == test;
17
18     ASSERT_EQ(btest, false);
19
20 }

```

Listing 10. Unit test example

APPENDIX C DATA MANAGEMENT

```

1
2 void DataRecorder::SaveDataRecord(const DataRecord&
   Record)
3 {
4     nlohmann::ordered_json root;
5
6     std::ifstream inFile(CorePaths::Contents.Path +
       SaveLocation.data());
7
8     CopyFileContents(root, inFile);
9
10    if (!root.is_array())
11    {
12        root = nlohmann::json::array();
13    }
14
15    size_t index;
16
17    if (DoesJsonContainRecord(Record, root, index))
18    {
19        AppendRecordToData(Record, root, index);
20        return;
21    }
22
23    CreateNewDataEntry(Record, root);
24
25 }
26
27 DataRecord record =
28 {
29     "testcard",
30     "testOS",
31     "TestRam",
32     "TestCPU",
33     "OpenGL",
34     "Test10",
35     "Test100",
36     "Test1000"
37 };
38
39 DataRecorder::SaveDataRecord(record);
40

```

Listing 11. The code to save data to a json file

```

1 #Loads the data, and normalises it into a format
   that cen be used for testing
2 def LoadData(path):
3     data = pd.read_json(path)
4     data = pd.json_normalize(data["Data"])
5     return data
6
7 #Runs the Wilcoxon-Mann Whitney test, returning the
   results
8 def RunTest(vulaknTimes, openGLTimes):
9     return mannwhitneyu(vulaknTimes, openGLTimes)
10
11 #Creates a box plot from the vulkan times and opengl
   times, Only for one set
12 def CreateBoxPlot(vulkan_data, opengl_data):
13
14     plot_df = pd.DataFrame({
15         "Time taken": vulkan_data + opengl_data,
16         "API": ["Vulkan"] * len(vulkan_data) + ["
OpenGL"] * len(opengl_data)
17     })
18
19     plot_df.boxplot(column="Time taken", by="API")
20
21     plt.title("Time taken by API")
22     plt.suptitle("")
23     plt.show()
24     return
25
26 def main():
27
28     data = LoadData("TestData.json")
29
30     Key = input("Set to test: 10, 100, 1000")
31
32     vulkan_api = data[data["API"] == "Vulkan"][Key].
       iloc()[0]
33     vulkan_opengl = data[data["API"] == "OpenGL"][
       Key].iloc()[0]
34
35     test = RunTest(vulkan_api, vulkan_opengl)
36
37     file = open("WhitneyTestResults.txt", "a")
38     file.write("Set: " + str(Key) + " ,statistic: "
       + str(test.statistic) + " , p_value: " + str(test
       .pvalue) + "\n")
39     file.flush()
40
41     CreateBoxPlot(vulkan_api, vulkan_opengl)

```

Listing 12. Creating a box plot from data and recording the Wilcoxon-Mann Whitney test.

APPENDIX D ETHICS APPROVAL


Research ethics approval
 Approved, subject to confirmation that machines used are authorized

Fig. 7. Ethics approval

APPENDIX E
PROJECT PLAN

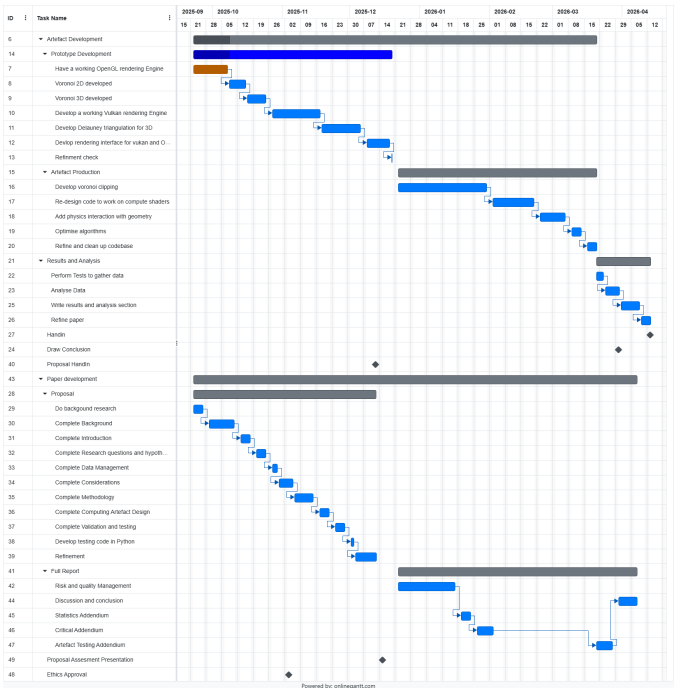


Fig. 8. A gantt chart showing the current project plan